

Student name: Le Trung Quoc Tran

Student Number: C3505289

Assignment 1 Report

Results after I ran four algorithms on the same input as the datafile1.txt:

Summary		
Algorithm	Average Turnaround Time	Waiting Time
FCFS	16.40	12.60
NRR	15.80	12.00
HRRN	16.20	12.40
PAP	17.20	13.40

- Comparing the results:

Looking at the results table, I notice that NRR gives the best numbers overall, with an average turnaround of 15.80 and an average waiting time of 12.00. This is easy to understand once you look at how NRR works: each process has its own quantum, starting at 4, and keeps getting cut off if it hasn't finished yet. Because of this, a short process like p2, whose service time is only 1, doesn't have to sit behind p1, whose service time is a full 10, the whole time, it gets slotted in and runs much sooner. You can see this clearly in the log, p2 finishes with a turnaround of only 7, while under FCFS it takes 13. This matches exactly what we learned in class about round robin, cutting the CPU time into smaller pieces reduces the waiting time for short jobs.

FCFS goes the other way, giving the worst result among the three non-priority algorithms, with an average turnaround of 16.40 and an average waiting time of 12.60. This also matches the theory we covered on the convoy effect, meaning that if a long process runs first, every short process behind it gets stuck waiting even though they only need a tiny amount of time to finish. p1 running for the first 10 units means p2, p3, p4 and p5 all have to wait, including processes that only need 1 or 2 units of time.

HRRN sits in the middle, doing a bit better than FCFS, with an average turnaround of 16.20 and an average waiting time of 12.40. Since all five

processes arrive at the same time, with ArrTime equal to 0, the response ratio formula ends up roughly favoring the shorter job, so p2 and p4 get pulled ahead of p1. But HRRN is still non preemptive, meaning once a process is chosen to run it has to run to completion, there's no flexibility like in NRR, which is why it still falls behind NRR overall.

PAP I found genuinely interesting, because it doesn't follow the run as fast as possible approach the other three do, it focuses instead on respecting the priority that was assigned, with an aging mechanism so a low-priority process is never starved forever. In this dataset, p5 has the lowest priority, a value of 5, and a large aging interval, also 5, so it ends up waiting the longest, and that's intentional, not random. Seeing the average turnaround of 17.20 and waiting time of 13.40 come out higher than the other three, it's easy to assume that's a worse result, but this is just the nature of priority scheduling, accepting a trade off in average waiting time in exchange for keeping the correct priority order. In other words, PAP isn't aiming for the same goal as FCFS, HRRN or NRR, so comparing the raw averages directly is a bit of an uneven comparison, but it still shows the tradeoff between priority-based fairness and average processing speed clearly.

- Trends and anomalies:

+ The main thing I had to fix was FCFS. I originally wrote it using plain formulas, basically doing the maths directly from ArrTime and SrvTime to get the start, turnaround and waiting time, and it worked fine on its own. But once I started HRRN, NRR and PAP, I realized those algorithms need to track the ready queue and the current time step by step, since you can't just write one closed formula for them because the order processes run in depends on what's happening in the queue at each moment. So, I went back and rewrote FCFS to use the same CPU and queue simulation, meaning admit, dispatch, run and finish, as the other three, using a shared Scheduler interface, instead of leaving it as a separate special case. This matches how we were taught to think about scheduling in the lab sessions.

```
J FCFS.java 1 X
C: > Users > trung > OneDrive > Desktop > úc > Nam Hai > OS > A1 làm > J FCFS.java > FCFS > startTime(int)
1  import java.util.*;
2
3  public class FCFS {
4      private List<Process> processes;
5
6      public FCFS(List<Process> processes) {
7          this.processes = processes;
8      }
9
10     private Process findByPid(int pid) {
11         for (Process p : processes) {
12             if (p.getPid() == pid) return p;
13         }
14         return null;
15     }
16
17     public int startTime(int pid) {
18         Process current = findByPid(pid);
19
20         if (pid == 1) {
21             return current.getArrTime() + current.getDisp();
22         }
23
24         Process prev = findByPid(pid - 1);
25         return startTime(pid - 1) + prev.getSerTime() + current.getDisp();
26     }
27
28     public int waitTime(int pid) {
29         Process current = findByPid(pid);
30         return startTime(pid) - current.getArrTime();
31     }
32
33     public int turTime(int pid) {
34         Process current = findByPid(pid);
35         return current.getSerTime() + waitTime(pid);
36     }
37 }
```

(FCFS version 1)

+ The other issue I found was in PAP: when two processes had the exact same effective priority, my code just kept whichever one it happened to compare first, instead of following a clear rule. This gave the wrong dispatch order once during testing. I fixed it by adding a tie break rule that picks the smaller pid when priorities are equal, the same rule I was already using in HRRN, and after that the output matched the expected schedule exactly.

=> Conclusion: Overall, the results match what we covered in class: NRR, which is preemptive and uses a small quantum, gave the lowest average waiting time, FCFS suffered the most from the convoy effect, HRRN improved on FCFS a bit by favoring shorter jobs, and PAP traded off average waiting time for priority fairness with aging. Building all four algorithms on top of one

shared CPU and queue simulation instead of separate formulas made the code much easier to fix and compare once I hit the FCFS problem above.